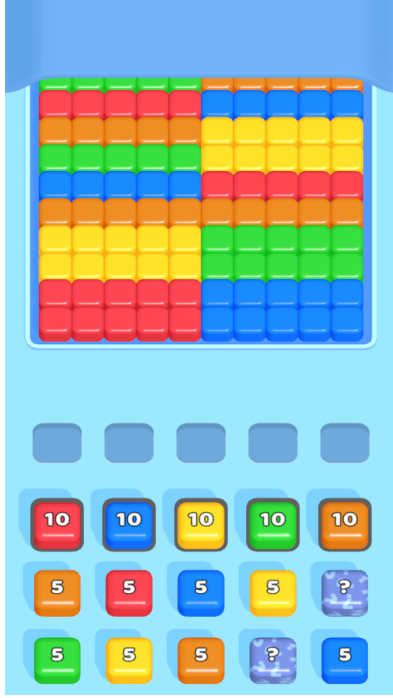


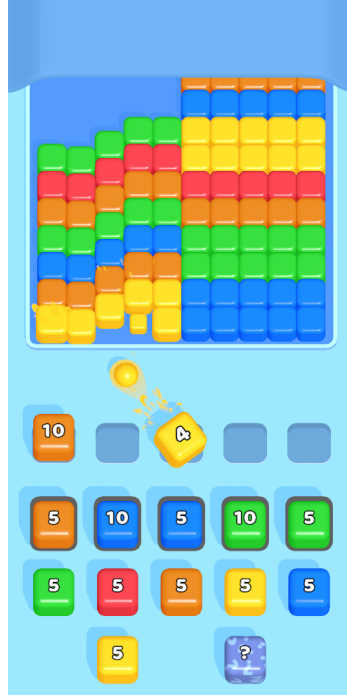
This Is Blast

Apps Game Developer Case · Salih Gireniz · 1-7 Eylül 2026

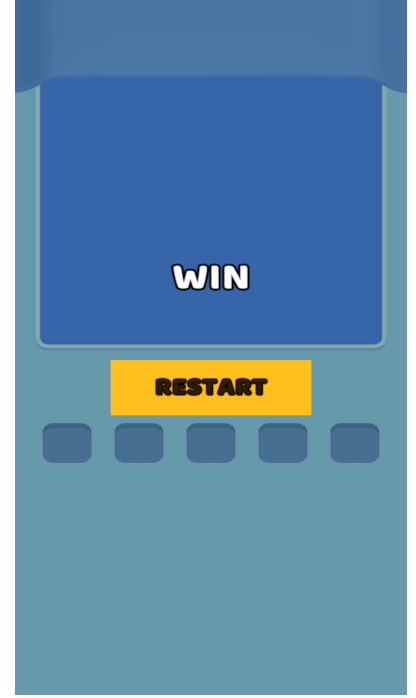
Voodoo'nun *This is Blast!* çekirdek döngüsünün sıfırdan yapımı: 10x10 küp ızgarası, beş slot, ön sırası tıklanabilir bir shooter kuyruğu, gizli shooter'lar, WIN / LOST ekranları ve aynı leveli yeniden başlatan buton, JSON'dan okunan bir örnek level, merge yok. Unity 6000.0.68f1, URP 17. Sahne açılıp Play'e basıldığında level doğrudan başlar; menü yok.



Açılış: örnek level, iki gizli shooter



Üç shooter ateşte: mermi, iz ve sıçrama shooter'ın renginde



WIN ekranı ve aynı leveli yeniden başlatan buton

1. Brief'te istenenler ve karşılıkları

Brief	Projede
10x10 ızgara, 5 slot, level'in belirlediği shooter kolonları, seçilebilir sıranın arkasında 2 görünür sıra	LevelSpawner; her yerleşim sayısı serileştirilmiş bir struct
Ön sıradaki shooter'a dokun; boş slotu koşar, kendi renginin ön küplerine ateş eder	GameDirector → GameLoop.TrySelect / TryShoot
Yıkılan küpün arkasındakiler shooter'a doğru akar	BoardModel ön indeksi + CollapseTweens
Mühimmat bitince sahneden çıkar, slotu boşaltır; mühimmat var hedef yoksa yerinde kalır	SlotRow: doluluk mühimmatın kendisidir
Gizli shooter: rengi seçilebilir sıraya gelince görünür	ShooterQueue.IsRevealed, LevelSpawner
WIN / LOST ekranı ve aynı leveli yeniden başlatan buton	LevelEndViewModel (R3) + LevelEndView, SceneRestarter
JSON'da tek örnek level: beş renk, gizli shooter, çözülebilir	Level_01.json; LevelFileTests her level dosyasında renk başına mühimmat ≥ küp kuralını doğrular
Merge yok	Yapılmadı; kancası da yok

Oyun hissi (juice), orijinal APK'dan kare kare ölçülüp elle ayarlandı: shooter'ın koşusu, iniş ezilmesi ve sayaç vuruşu, hedefe dönüş, shooter renginde izli mermi, küpün üst yüzünde sıçrama, çizilen ölüm eğrisi, kamera sarsıntısı, iki sesli ve perde titreşimli atış sesi. Küpler darbeyi alır: hedef ve merminin yolda içinden geçtiği her ön küp, temas noktası etrafında savrulup o noktadan uzağa itilir ve tasarımcının eğrisiyle yerine döner (FlightPath temas noktasını collider olmadan bulur; CubeView.Nudge küp başına tek yeniden kullanılan tween'dir, atış başına bellek ayırmaz).

2. Mimari

Altı assembly. Bağımlılık yönü asmdef referanslarıyla zorlanır, gelenekle değil; ArchitectureTests asmdef dosyalarını diskten okur ve bir katman üstündekine referans verirse kırmızıya döner.

Blast.Domain	saf C#: BoardModel, ShooterQueue, SlotRow, GameRules (noEngineReferences)
^	
Blast.Application	GameLoop: TrySelect / TryShoot, karar, kolon durumları
^	
Presentation	UI (MVVM, R3) Infrastructure (LevelParser, PaletteData)
^	
Blast.Bootstrap	GameLifetimeScope: VContainer kompozisyon kökü
Blast.Diagnostics	PerfProbe / PerfSweep; yalnız development build, oyun katmanına referans yok

- **Domain** Unity'yi bilmez ve içinde hiçbir şey hareket etmez: kart yazıldığı renkleri tutar, kolon başına bir ön indeks üzerinde yürür; kuyruk aynısını yapar. Bir slot, shooter'ın mühimmatı olduğu sürece doludur; "boş slot" ile "mühimmat bitti" birbiriyle çelişemez. GameRules üç saf fonksiyondur: en soldaki eşleşen ön küp, kazanıldı mı, kaybedildi mi.
- **Application** tek kullanım senaryosudur: GameLoop. Bir çağrı bir atışı bir küple değiştirir ve kararı aynı çağrıda yeniden okur; ekran hiçbir zaman bir hamle geç kalmaz. Önce *kazanıldı*, sonra *kaybedildi* sorulur, çünkü boşalmış kart ve dolu slot sırası ikisini de sağlar. Zaman kavramı yoktur; tempoyu Presentation verir.
- **Presentation** döngünün cevaplarını giydirir. GameDirector dokunuşu TrySelect'e çevirir ve her oturan shooter'ın atış döngüsünü UniTask ile yürütür; LevelSpawner dünya konumlarının tek sahibidir; view'lar itaatkârdır, karar vermez. Kolon durumu el sıkışması (LockColumn / MarkSettling / MarkSettled), domain küpü vurulduğu anda silerken merminin ekranda hâlâ havada olmasından doğar.

- **UI** R3 akışları üzerinde tek bir MVVM çifti. **Infrastructure** güven sınırır: parser her bozuk dosyayı yerini söyleyen bir `FormatException` ile reddeder.
- **Bootstrap** parse edilmiş modelleri, `GameLoop`'u, palet adaptörünü ve sahne bileşenlerini kaydeder; `VContainer` grafiği kurar ve her bileşenin `[Inject]` `Construct`'ını çağırır.

3. Testler

Takım	Sayı	Çalıştırma
Unity EditMode, assembly <code>Blast.Tests</code>	109	Test Runner penceresi
Level editörü, <code>level-editor/logic.js</code>	41	<code>node --test level-editor/logic.test.js</code>

Oyunun her kuralının testi var; `MonoBehaviour`'lar `Humble Object` deseniyle test edilir. Sessizce kırılan iki şeyi iki test korur: `asmdef` bağımlılık yönü ve her level dosyasında renk başına mühimmat $\geq$ küp kuralı (brief'in uyardığı "kırmızı küp var, kırmızı shooter yok" leveli).

4. Level verisi ve orijinal oyunun okunması

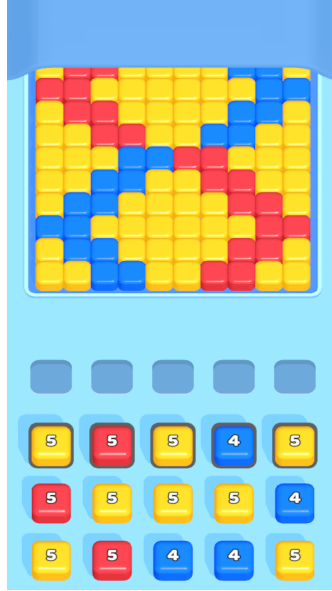
Level bir metin dosyasıdır: sıralar ön sıradan başlayarak kolon başına bir harf (Y R B G O), her shooter kolonu önden arkaya, hidden rengi ön sıraya kadar gizler. `level-editor/index.html` bağımlılıksız bir HTML editördür: kartı boyar, kuyruğu otomatik doldurur, testlerle aynı kontrolleri yapar.

```
{ "boardLayers": [ { "rows": [ "RRRRRBBBBB", "..."] } ], "slotCount": 5,
"shooterColumns": [ { "shooters": [ { "color": "R", "ammo": 10, "hidden": false } ] } ] }
```

Orijinal APK okundu, kopyalanmadı

Yayınlanan build `UnityPy` ile açıldı; kural: **sayı oku, asset alma**. Görünümün nelerden oluştuğu ölçüldü (Linear renk uzayı, küp tonları, TCP2 ramp değerleri, gölge bayrakları) ve 2447 levelin nasıl saklandığı çıkarıldı: kart ve deste sütun-major, kartın son sırası shooter'lara bakar, gizli shooter bir `SecretItem` bayrağı, mühimmat her shooter için sabit 20. Levellerin 6 A/B kohortuna ayrılması (onboarding, 7. gün, erken churn) orijinalin canlı-operasyon yüzeyini de gösterdi.

`Docs/Tools/convert_original_level.py` bu şemayı bizimkine çevirir ve sonucu bizim kurallarla ispatlar: oyuncunun seçimleri üzerinde bir arama (DFS) kazanan bir hat bulmalı, rastgele atış sırası altında yeniden planlayan oyuncu kazanmalı. Orijinalin 4. leveli olduğu gibi girdi (`Level_Original_04.json`; 10x12, kendi renkleri, çalışan APK ile küp küp karşılaştırıldı) ve çözücü onu gerçek oyunda 26 dokunuşta kazandı.



Orijinalin 4. leveli bu projede: ön sıra
YYBBYYRRYY, orijinalle aynı

Yoğun leveller neden yok

Orijinal her shooter'a 20 mühimmat verir ve slotları merge ile açar; bu vakada merge yasak. Bizde mühimmat tam-uyumludur (renk başına küp sayısı o rengin shooter'larına bölünür), yoksa rengi biten bir shooter slotunu sonsuza dek tutar. Bu kurallar altında orijinalin yoğun levelleri ya çözülemez (18, 650) ya da yola bağımlıdır: 868 simülatörde %100 kazanılırken gerçek oyunda 12. dokunuşta takıldı, çünkü aynı renkten hangi shooter'ın ateş edeceğini kolon kilitleri ve koşu süresi belirler. Ölçüm üç kez aynı sonucu verdi; o yüzden örnek level elle yazılmış Level_01 olarak kaldı. Brief'in "tutarlı level" kuralının somut hali budur: başka kurallar için yazılmış bir level, gerçek oyunda ölçülmeden tutarlı sayılmaz. Hiçbir mesh, doku, ses veya kod alınmadı.

5. Level editörü

Levelleri elle JSON yazarak değil, bir editörle yapıyoruz. Bu editör Unity'nin içinde değil; deponun kökünde, tarayıcıda çalışan bir HTML sayfası. Kartı fırçayla boyar, shooter kuyruğunu kolon kolon düzenler ve JSON'u doğrudan Unity'nin okuduğu klasöre yazar. Kullanıcıları tasarımcılar olduğu için arayüzü Türkçe.

Nerede

Depo kökünde level-editor/ klasörü, üç dosya: index.html (arayüz), logic.js (level mantığı: okuma, yazma, doğrulama, otomatik doldurma, simülasyon) ve logic.test.js (41 test). Assets/ dışındadır, Unity onu hiç görmez ve derlemez; oyuna tek dokunuşu, Assets/00_GAME/Levels/ klasörüne yazdığı JSON dosyalarıdır. Oyun o dosyaları kendi parser'ından geçirir; editör ne yazmış olsa da son söz oyunun.

Neden HTML

Alternatif bir Unity Editor penceresiydi: C# editör script'i, derleme, domain reload, ve level yapmak isteyen herkesin Unity'yi açıp projeyi yüklemesi. HTML'in maliyeti sıfır: kurulum yok, paket yok, sunucu yok, internet gerekmez; dosya çift tıkla açılır ve Unity kapalıyken de level yapılır. Level formatı zaten düz JSON olduğu için editörün Unity'den bir şey bilmesi gerekmiyor; aynı kuralları (renk başına mühimmat, kolon sayısı, boş kolon) JavaScript'te uygular ve bunlar node --test ile ayrı bir test takımında tutulur. Tek gerçek bedel, tarayıcının klasöre yazma izni: o da aşağıda.

Nasıl açılır

Dosya yöneticisinde (Windows'ta Gezgin, macOS'te Finder) level-editor/index.html dosyasına çift tıklayın ya da dosyayı açık bir tarayıcı sekmesine sürükleyin; adres çubuğunda file:///.../level-editor/index.html görünür. Her tarayıcıda açılır ve level yapılır. Klasöre doğrudan

yazma yalnız Chromium tabanlı tarayıcılarda vardır (Chrome, Edge, Opera, Brave); Safari ve Firefox'ta dosya **İndir** ile iner ya da level **JSON kopyala / JSON yapıştır** ile pano üzerinden taşınır. Sayfa açıldığında kırmızı bir uyarı hangi klasörün seçileceğini söyler; ilk iş o. Kafa karıştırabilecek üç noktayı sırayla yazıyoruz: klasör seçimi, kaydetme ve Unity'nin dosyayı görmesi.

1. Klasör seçimi

Editör dosyaları tarayıcının klasör erişimi (File System Access API) ile yazar; bu yalnız Chromium tabanlı tarayıcılarda vardır (Chrome, Edge, Opera, Brave), Safari ve Firefox'ta yoktur. Açılışta üstte kırmızı bir uyarı, seçilmesi gereken klasörün tam yolunu gösterir (...\\Assets\\00_GAME\\Levels) ve yanında **Yolu kopyala** düğmesi vardır. **Levels klasörünü seç...** düğmesine basınca tarayıcının klasör penceresi açılır; yolu üstteki adres alanına yapıştırıp Enter'a basın, klasörün *içine* girin ve **Select Folder** deyin. Tarayıcı bir kez izin ister, sonraki açılışlarda klasörü hatırlar. Dikkat: Edge kullanıcı klasörünün kendisini (Masaüstü, Belgeler) reddeder; seçilen klasör Levels'ın kendisi olmalı. Klasör seçildikten sonra üstteki liste klasördeki levelleri gösterir; **Aç** ile ya da çift tıklıya yüklenir. Klasör erişimi olmayan bir tarayıcıda editör yine çalışır: **Aç** bir dosya penceresi açar, **İndir** JSON'u indirir, dosyayı Levels klasörüne elle atarsınız. İki düğme de her tarayıcıda çalışır: **JSON kopyala** levelin JSON'unu panoya alır (bir .json dosyasına yapıştırmak ya da bir leveli mesajla göndermek için), **JSON yapıştır** panodaki ya da elle yapıştırılan JSON'u level olarak açar ve aynı kontrollerden geçirir.

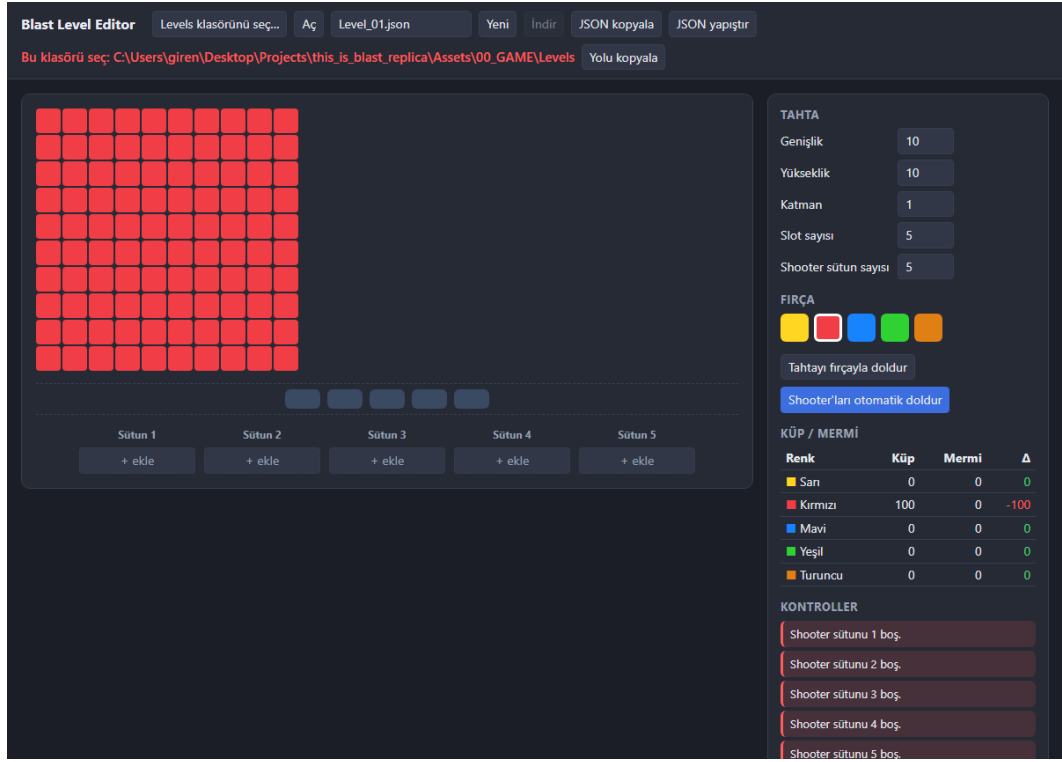
2. Yeni level ve var olanı güncelleme

Yeni, listede boş olan ilk adla (Level_07.json gibi) 10x10, tek katman, beş slot, beş kolonluk boş bir level açar. Sağdaki panelden genişlik, yükseklik, katman ve kolon sayısı değişir; fırçadan renk seçip hücrelere tıklayarak ya da **Tahtayı fırçayla doldur** ile kart boyanır. Her kolonun altındaki **+ ekle** shooter ekler; her shooter'ın rengi, mermisi, gizli (?) kutusu ve yukarı/aşağı oklarıyla kuyruk sırası vardır. **Shooter'ları otomatik doldur** karttaki renk sayımına göre kuyruğu tam-uyumlu mühimmatla dağıtır; sağdaki Küp / Mermi tablosu renk başına farkı (Δ) anında gösterir, kırmızı bir Δ kazanılamaz bir level demektir. Kontroller bölümü Unity testleriyle aynı kuralları uygular ve her birini yerini söyleyerek yazar: boş tahta, boş kolon, mermisiz shooter, küpünden az mermisi olan renk ("level kazanılamaz"), beşten fazla kolon (dock'a sığmaz). Üstüne bir de ağgözlü bir simülasyon oynar: hep en soldaki hedefe ateş eden basit bir oyuncu leveli bitiriyorsa "Hata yok" der, takılıyorsa kaç küp kala takıldığını uyarı olarak gösterir; dikkatli bir oyuncu yine kazanabilir, ama kuyruk sırasına bakmak gerekir.

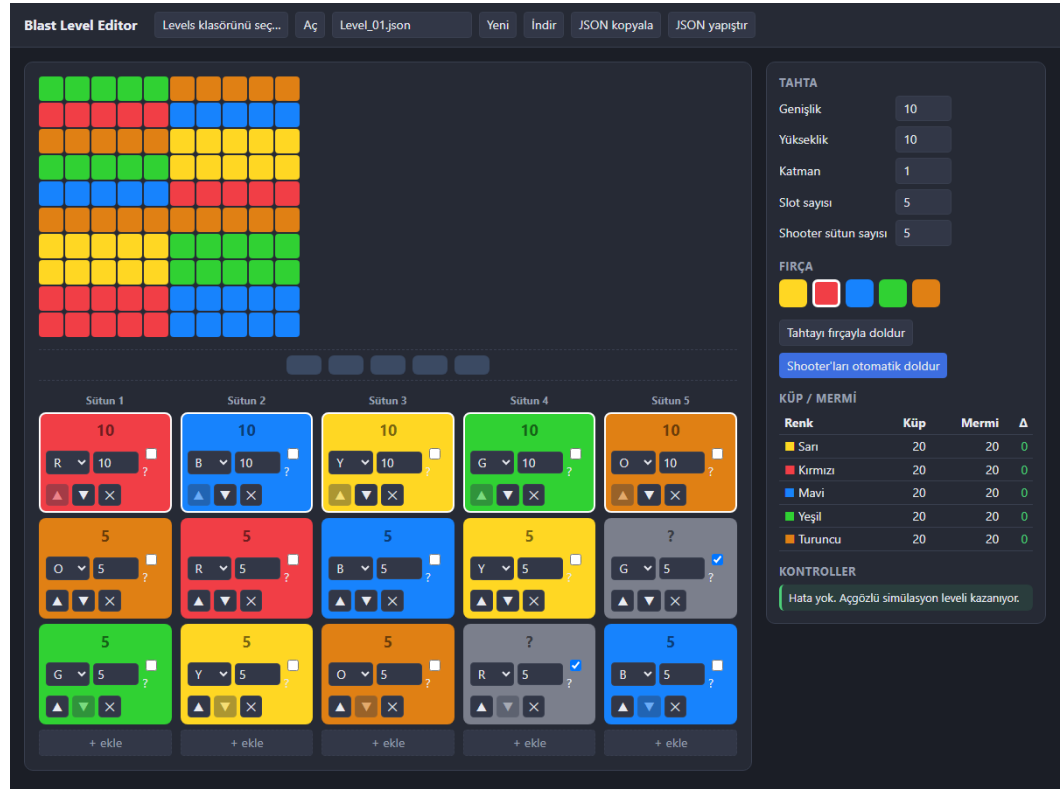
Kaydet açık dosyanın üstüne yazar; **Farklı Kaydet** soldaki ad alanındaki adla yazar ve var olan bir dosyanın üstüne yazmadan önce sorar. Var olan bir leveli güncellemek için listeden açın, değiştirin, Kaydet. Kaydedilmemiş değişiklik varken başka dosya açmak sorar. Editörün gösteremediği bir şey içeren dosya (farklı renkli katmanlar gibi) açılırsa üstüne yazmaz, başka ad ister.

3. Unity'nin dosyayı görmesi

Editör JSON'u diske yazar; Unity ise değişen bir dosyayı kendi penceresi odak aldığı anda içe aktarır. Çoğu zaman bu yeterlidir: editörden kaydedip Unity'ye geçin, Play'e basın. Ama Unity'nin otomatik yenilemesi kapalıysa ya da odak değişimini yakalamazsa eski level oynanır ve "kaydettim ama değişmedi" hissi doğar. O durumda iki yol var: Unity penceresinde **Ctrl+R** (Assets → Refresh), ya da Project panelinde Level_XX.json dosyasına sağ tık → **Reimport**. Level_01 Play'de doğrudan açıldığı için onu düzenledikten sonra Play'i durdurup yeniden başlatmak gerekir; başka bir leveli oynamak için sahnedeki GameLifetimeScope bileşeninin _level alanına o dosyayı sürükleyin.



Açılış: kırmızı uyarı seçilecek klasörün yolunu gösterir, Yolu kopyala düğmesi yanında; sağda kart ayarları, fırça, Küp / Mermi tablosu ve kontroller

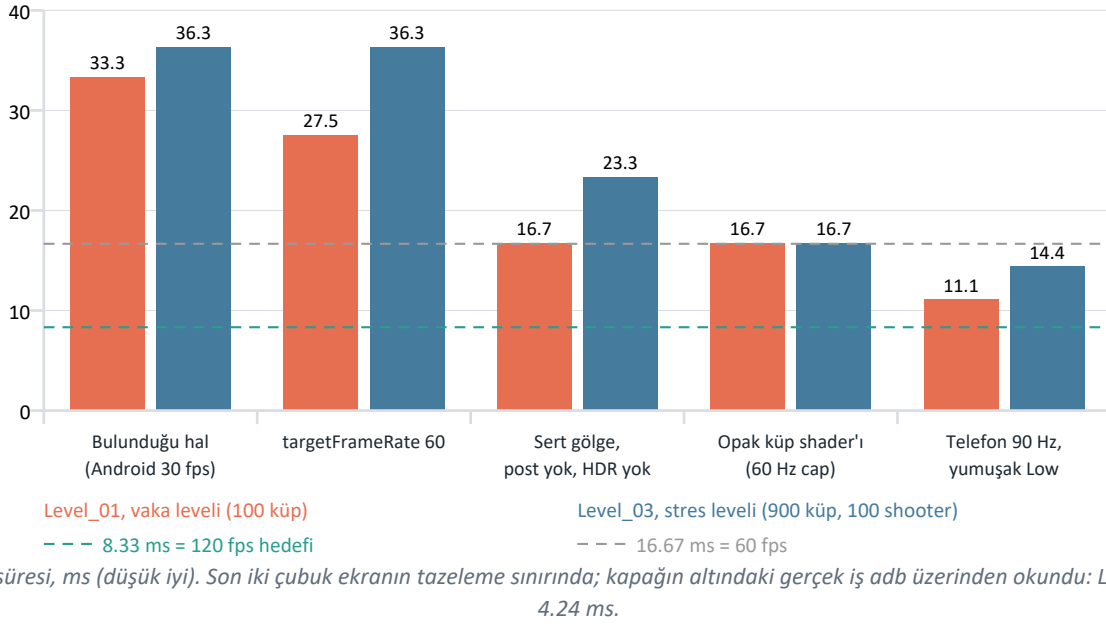


Level_01 açık: kart boyalı, beş kolonda shooter'lar (renk, mermi, gizli kutusu, sıra), Küp / Mermi tablosunda her Δ sıfır, Kontroller yeşil

6. Performans

Hedef: 120 FPS ve oyun sırasında kare başına 0 B GC. Her ölçüm Samsung Galaxy A16 üzerinde, adb ile yüklenen development build'de, saniyede bir satır yazan bir çalışma zamanı probuyla alındı (fps, kare süresi, GC bayt, batch, SetPass, draw, gölge kaynağı, üçgen). Editör sayıları render ve GC için kullanılmadı; yalnız yanıltırlar.

Yöntem hep aynıydı: önce ölç, tek bir şeyi değiştir, tekrar ölç, sonucu deftere yaz. Defter Docs/PERFORMANCE.md, alınmayanlar da sayısıyla orada.



Nereye gitti, ne yapıldı

İlk okuma varsayımı tersine çevirdi: darboğaz draw sayısı değil, gölge örnekleme idi. Yumuşak gölge High kalitede piksel başına 16 doku okuması yapıyor ve küpler ekranı kaplıyor; sert gölge tek başına iki levelde de 8 ms geri verdi. İkinci büyük kalem küp shader'ıydı: TCP2'nin ürettiği CustomShader TransparentCutout'tu ve her pikselde clip çağırıyordu; oyunda alfası olan tek doku yok, ve Mali GPU'da bir fragment clip early-Z'yi öldürür. Opak yapılip clip'ler silindi: 900 küplük levelde 7 ms. Post-process (vignette) ve HDR bir ara doku ve blit demekti, ölçüldü ve 2-3 ms için kaldırıldı. targetFrameRate Android'in varsayılan 30'undan 120'ye alındı; telefon ekranı 90 Hz olduğu için gerçek tavan 11.1 ms.

CPU tarafında hedef sıfır tahsisti. Kaynaklar tek tek kapatıldı: hareket eden her view için her hamlede yeni bir DOTween kısayolu yerine bir kez kurulan ve yeniden hedeflenen tek tween (SetAutoKill(false) + ChangeEndValue), küp ölümleri için paylaşılan bir çökme tween havuzu, mermi ve sıçrama havuzları, iki sesli sabit bir ses kanalı halkası, her yerde sharedMaterial (renk değişimi materyal örneği değil, paletin hazır materyaline geçiş). Boşta 16 B/kare bulunan tek tahsis Easy Save'in bir coroutine'iydi; eklenti silindi. Sonuç: boşta 0 B, ateş ederken salvo başına ~11 B/kare oyun tahsisi, o da CancellationToken.Register'ın restart güvenliği için bilerek bırakılan 48 baytları.

	Bulunduğu hal	Teslim edilen
Level_01 kare süresi	33.3 ms (30 fps)	11.1 ms, telefonun 90 Hz sınırı; altında 4.24 ms gerçek iş
Level_03 (900 küp) kare süresi	36.3 ms	14.4 ms
SetPass call	11-14, her sahnede	11-14, değişmedi
GC, boşta	16 B/kare	0 B
GC, ateş ederken	78 B/kare	0 B; bir salvo içinde ~11 B/kare oyun tahsisi

Neden SRP Batchter'a güvenildi

İlk ölçümde SetPass sayısı küp sayısından bağımsız 11'de duruyordu: 100 küp de 900 küp de aynı. Bu, URP'nin SRP Batchter'ının işini yaptığının kanıtıdır. Batchter, aynı shader varyantını kullanan her renderer'ı materyali farklı olsa bile tek bir batch'te toplar ve materyal verilerini GPU'da kalıcı tutar; küplerin hepsi aynı TCP2 shader'ının aynı

varyantı olduğu için beş renk beş SetPass etmiyor. Mermi de küplerin palet materyalini giyer, o yüzden atış sırasında draw sayısı artmaz; renkli parçacık da beyaz parçacıkla aynı draw'dur. Buna güvenmenin şartı, batcher'ı kıran şeyleri yapmamaktır: MaterialPropertyBlock yok, çalışma zamanında materyal örneği yok, per-renderer keyword yok.

Neden GPU instancing yapılmadı

Instancing ile SRP Batchter birbirini dışlar; instancing'e geçmek batcher'dan vazgeçmek demektir. Karar tahminle değil ölçümle verildi: draw sayısının maliyetini görmek için 390 küp gölge geçişinden çıkarıldı, 390 batch gitti ve kare süresi 0 ms değişti. Draw sayısı hiçbir zaman darboğaz olmadı; SetPass zaten 11-14'te. Instancing'in kazandıracağı şeyi batcher zaten kazandırıyor, üstüne bir de kendi shader desteğini ve per-instance veri yönetimini getirecekti. Yapılmadı, gerekçesi sayısıyla defterde.

Neden küpler havuzlanmadı

Havuz, dönen (churn eden) nesneler için vardır: mermi ve sıçrama saniyede kırk kez doğup ölür, bu yüzden havuzlandılar. Bir küp level yüklenirken bir kez yaratılır, ölürken bir kez yok edilir; dönme yok. Havuz maliyeti yükleme anından yükleme anına taşır ve yönetilecek bir ömür ekler; tahsis avında ölüm yolu ölçülebilir hiçbir şey katmıyordu (salvoda toplam 11 B/kare). Aynı şekilde ekran dışı küpleri kapatmak da ölçüldü: 400 küpün 130'u frustum içindeydi ve Unity zaten 378 batch çiziyordu, yani frustum culling işi yapıyordu; elle kapatmak sadece Unity'nin bedava yaptığı testi C#'a taşımak olurdu ve ışığa göre kesilen gölge kaynaklarını yanlışlıkla düşürdü. İkisi de alınmadı.

Ölçülüp alınmayan diğerleri: 512'lik gölge haritası (0 ms), MSAA'yı kapatmak (0 ms), render scale 0.75 (2.5 ms ama bulanık), Optimized Frame Pacing (+2 ms, 90 Hz'e geçirmede). Kapağın altındaki gerçek iş adb üzerinden profiler ile okundu: Level_01'de kare başına 4.24 ms, tamamı render; listede hiçbir gameplay script'i yok. 11.1 ms'lik bütçeye karşı 2.6 kat pay; oyun ekrana bağlı, hesaba değil, ve iş burada bitti.

7. Süreç ve git

1-6 Eylül arasında 150'den fazla commit; her yeşil parça bir commit, her davranış önce testiyle. Değerlendirme sırası (bug-free > oyun hissi > mimari > performans > git) her çelişkide karar vericiydi: bir bug riski taşıyan juice eklenmedi, teslimden günler önce orkestratörü yeniden yazmak reddedildi. Depoda üç kayıt dosyası vardır: Docs/PLAN.md (tasarım), Docs/PERFORMANCE.md (performans defteri), Docs/ORIGINAL_GAME_ANALYSIS.md (orijinalin okunması); parça parça yapım kaydı .claude/notes/case-status-log.md'dir.